

CE 310

Python Tutorial — Hypothesis Testing

Week 11 · Is the difference real, and is it big enough to act on?

Today's Plan — Four Tests, One Question

By the end of this walkthrough you will know how to:

1. Run a **two-sample t-test** with `scipy.stats.ttest_ind`
2. Compute **Cohen's d** (effect size) by hand
3. Run a **one-sample t-test** with `scipy.stats.ttest_1samp`
4. Run a **one-way ANOVA** with `scipy.stats.f_oneway`
5. Run a **Tukey HSD** post-hoc test with `pairwise_tukeyhsd`

Assumed from Weeks 9–10: you can load a CSV, filter rows, and read statsmodels output.

Files needed: `CE310_BidItems_2024_2026.csv` and `CE310_BidProjects_2024_2026.csv`

The Data — What a Bid Tabulation Is

When a public agency advertises a job, every contractor bids the **same list of items**. The agency publishes the whole table afterwards.

Column	What it holds
UnitPrice_USD	what this contractor offered to charge per unit
EngineerEst_USD	what the agency's own engineer estimated
Quantity	how much of the item the job needs
LowBidder	did this contractor win the contract
District	which of 25 TxDOT districts let the work

💡 Everyone bids the same item on the same job, so a difference between two contractors is a difference in *pricing*, not in the work.

Step 1 — Setup

 No new libraries beyond Week 10 — `scipy.stats` and `statsmodels` are already loaded.

```
import pandas as pd
import numpy as np
from scipy import stats
from statsmodels.stats.multicomp import pairwise_tukeyhsd
import statsmodels.formula.api as smf

items = pd.read_csv('CE310_BidItems_2024_2026.csv')

# One line item, so every comparison is like-for-like
exc = items[items['ItemDescription'] == 'EXCAV (ROADWAY)'].copy()
exc['Ratio'] = exc['UnitPrice_USD'] / exc['EngineerEst_USD']
```

Step 1 — Split Winners from the Rest

```
low  = exc[exc['LowBidder'] == 'Yes']['UnitPrice_USD']
rest = exc[exc['LowBidder'] == 'No']['UnitPrice_USD']

print(f'winners  n={len(low):,}  mean=${low.mean():.2f}  median=${low.median():.2f}')
print(f'others   n={len(rest):,}  mean=${rest.mean():.2f}  median=${rest.median():.2f}')
```

Expected: winners $n = 555$ · mean $\approx \$28.14$ · median $\$19.00$

others $n = 3,814$ · mean $\approx \$30.73$ · median $\$22.00$

⚠ Note the gap between mean and median in *both* groups. Hold that thought — Step 6 comes back to it.

Step 2 — Two-Sample t-Test

H_0 : the two group means are equal · H_1 : they differ · $\alpha = 0.05$

```
t_stat, p_val = stats.ttest_ind(low, rest, equal_var=False)
print(f't = {t_stat:.4f}    p = {p_val:.4f}')
```

Expected: $t \approx -1.71$ · $p \approx 0.087$

At $\alpha = 0.05$ you **cannot reject H_0** . The medians differ by \$3.00, but the test does not find it.

💡 `equal_var=False` runs Welch's t-test, which does not assume the two groups share a variance. Prefer it by default — you rarely know that they do.

Step 3 — Cohen's d (Effect Size)

A p-value says whether a difference is distinguishable from zero. It says nothing about whether the difference is **big**.

```
n1, s1 = len(low), low.std()
n2, s2 = len(rest), rest.std()
pooled_sd = np.sqrt(((n1 - 1) * s1**2 + (n2 - 1) * s2**2) / (n1 + n2 - 2))
cohens_d = (low.mean() - rest.mean()) / pooled_sd
print(f'pooled SD = {pooled_sd:.2f}    d = {cohens_d:.4f}')
```

Expected: pooled SD $\approx$ \$31.89 · d $\approx$ -0.081

d	reading
< 0.2	negligible
0.2–0.5	small
0.5–0.8	medium
> 0.8	large

Step 4 — One-Sample t-Test

Every project carries an **engineer's estimate** prepared before bids open.
A ratio of 1.0 means the bid landed exactly on it.

H_0 : mean ratio = 1.0 · H_1 : mean ratio $\neq$ 1.0

```
t_ratio, p_ratio = stats.ttest_1samp(exc['Ratio'], 1.0)
print(f"mean = {exc['Ratio'].mean():.4f}    median = {exc['Ratio'].median():.4f}")
print(f't = {t_ratio:.4f}    p = {p_ratio:.4e}')
```

Expected: mean $\approx$ 1.207 · median **exactly 1.000** · t $\approx$ 18.73 · p $\approx 2 \times 10^{-75}$

Step 5 — ANOVA Setup & District Groups

A t-test compares two groups. **ANOVA** compares many at once.

H_0 : all district mean unit prices are equal

```
groups = [g['UnitPrice_USD'].values
           for _, g in exc.groupby('District') if len(g) >= 30]
print(f'districts with n >= 30: {len(groups)}')
```

 The `if len(g) >= 30` guard drops districts too small to estimate a mean from. Say what you dropped when you report the result.

Step 5 — ANOVA Results & Tukey HSD

```
F_stat, p_anova = stats.f_oneway(*groups)
print(f'F = {F_stat:.4f}    p = {p_anova:.4e}')

tukey = pairwise_tukeyhsd(exc['UnitPrice_USD'], exc['District'], alpha=0.05)
res    = pd.DataFrame(tukey.summary().data[1:], columns=tukey.summary().data[0])
print(f'pairs = {len(res)}    significant = {(res["reject"] == True).sum()}')
```

Expected: $F \approx 14.53$ · $p \approx 8 \times 10^{-57}$ · 300 pairs · 74 significant

💡 ANOVA says *some* district differs. Tukey says *which*, while holding the error rate across all 300 comparisons — not across each one separately.

Step 6 — When the Mean Is the Wrong Summary

Two results from this walkthrough appear to contradict each other:

- **Step 4** rejected H_0 that the mean ratio is 1.0, overwhelmingly
- but the **median ratio is exactly 1.000**

```
print((exc['Ratio'] > 1.0).mean())    # share of bids above the estimate
print(stats.mannwhitneyu(low, rest)[1]) # rank test on the Step 2 groups
```

Expected: ≈ 0.409 above the estimate · Mann-Whitney $p \approx 7 \times 10^{-10}$

⚠ Step 2's t-test found nothing at $p = 0.087$. The rank test on the same two groups gives $p \approx 7 \times 10^{-10}$. Unit prices have a long right tail, and the mean carries it while the median does not. Look at the distribution before you trust a test about its mean.

Step 7 — Regression t-Values Revisited

Every regression coefficient carries a t-statistic testing H_0 : the coefficient is zero. Same machinery, applied to a slope.

```
exc['logq'] = np.log10(exc['Quantity'])
exc['logp'] = np.log10(exc['UnitPrice_USD'])
m = smf.ols('logp ~ logq', data=exc).fit()
print(m.summary().tables[1])
```

Expected: slope ≈ -0.167 · t ≈ -47.0 · $R^2 \approx 0.336$

💡 On a log–log scale the slope is an elasticity: 10× the quantity buys about a **32% lower** unit price, because $10^{(-0.167)} \approx 0.68$.

Quick Check — Before You Start the In-Class Exercise

Verify your numbers, then open `Wk11.03_In_Class_Exercise.ipynb`

Quantity	Expected
winners / others, n	555 / 3,814
two-sample t	≈ -1.71 ($p \approx 0.087$)
Cohen's d	≈ -0.081
one-sample t, ratio vs 1.0	≈ 18.73
ANOVA F across districts	≈ 14.53
Tukey pairs / significant	300 / 74
OLS slope, log price on log quantity	≈ -0.167

If any of these disagree, fix it **now** — the in-class exercise builds on all of them.

Quick Reference — Hypothesis Test Functions

Task	Call
Two-sample t-test	<code>stats.ttest_ind(a, b, equal_var=False)</code>
One-sample t-test	<code>stats.ttest_1samp(data, popmean)</code>
Rank test (no normality assumption)	<code>stats.mannwhitneyu(a, b)</code>
One-way ANOVA	<code>stats.f_oneway(*groups)</code>
Tukey HSD	<code>pairwise_tukeyhsd(values, labels, alpha=0.05)</code>

All return `(statistic, p_value)` except Tukey, which returns a results object — call `.summary()` on it.

Quick Reference — Effect Size

```
def cohens_d(a, b):  
    na, nb = len(a), len(b)  
    sp = np.sqrt(((na - 1) * a.var(ddof=1) + (nb - 1) * b.var(ddof=1))  
                  / (na + nb - 2))  
    return (a.mean() - b.mean()) / sp
```

Statistic	Answers
p-value	is the difference distinguishable from zero?
Cohen's d	is it big enough to change a decision?
F-statistic	do any of these groups differ?
Tukey HSD	which groups differ, corrected for testing many pairs

💡 With thousands of rows almost any difference becomes significant. The effect size is what tells you whether to act on it.

Before You Start

- [] Both CSV files uploaded to your Colab session
- [] Every expected value above reproduced
- [] `equal_var=False` used on two-sample tests
- [] You can say in one sentence why Step 2 and Step 6 disagree

Then open `Wk11.03_In_Class_Exercise.ipynb` and begin Section A.

Before You Submit

⚠ Following this submission format exactly is essential — with a class this size, grading depends on it. Submissions that don't comply will be penalized.

1. **Run all cells first.** In Colab: Runtime → Run all. Wait until every cell shows a checkmark — no ⏳ spinners remaining.
2. **Download as .ipynb.** File → Download → Download .ipynb. Do not submit PDF or .py.
3. **Do not edit the `print` lines.** Lines like `print(f"ANSWER_A2_t_stat = {ANSWER_A2_t_stat}")` must remain exactly as written — changing or deleting them means that part of your answer will not be counted.
4. **File name —** `CE310_W11_<NetID>.ipynb`, e.g., `CE310_W11_abc123.ipynb`.
5. **Upload** the `.ipynb` file to D2L Dropbox by **9:00 AM next Wednesday**.